

MMLU 自動化評估流程：跨模型評估與 AI 協同開發

建立一套可以公平比較不同 LLM、可重複執行、且容易驗證的 MMLU 評測流程。

01

240 題固定評測集

4 大領域 · 8 門學科
固定 seed = 42，抽樣條件可重現

02

3 模型跨維度比對

Gemini-3.1-Flash-Lite
對比 SmolLM2-1.7B、Qwen2.5-0.5B

03

全流程 AI 協作開發

VS Code + Cline
AI 實作 → AI 審查 → 人工驗收

單元測試：11 → 293 個測試案例 | 全套執行僅 4.91 秒

逐步建立完整測試防線，讓每次重構都有測試驗證，確保功能穩定與品質一致。

公平比較的第一步，是先固定評測條件。

8 門代表學科

每科 30 題

共 240 題基準

固定 seed = 42

相同輸入評測

4 大領域 | 各兩門學科

STEM college_computer_science | high_school_mathematics**Humanities** philosophy | world_religions**Social Sciences** econometrics | high_school_psychology**Other / Applied Sciences** clinical_knowledge | professional_law

評測規模

SMOKE TEST 80 題 | 每科 10 題 | 快速測試流程**DEMO BENCHMARK** 240 題 | 每科 30 題 | 正式比較

評測條件

0-SHOT 不給範例直接測，考驗模型「零樣本泛化與直覺答題能力」。**5-SHOT** 提供 5 題參考範例，考驗模型「上下文學習與格式遵循約束」。

系統架構：將資料、模型、評分彼此獨立，避免單點修改引發連鎖錯誤。

單向評測管線 Pipeline Data Flow

01

流程入口 `main.py`

透過命令列載入設定檔，統一啟動並執行完整 Benchmark 流程

02

資料準備 `dataset_loader.py`

下載MMLU、固定seed抽樣、組裝 0 / 5-shot Prompt；只管資料

03

排程與模型推論 `runner.py` → `src/models/`

循序載入本機 / 雲端；每題完成後即時寫入結果，降低中斷造成的資料遺失

04

答案評估 `evaluator.py`

從模型輸出中解析最終答案、計算 Accuracy 與偏差統計

YAML 集中管理實驗設定

`configs/eval_config.yaml`

單一設定來源

- 統一管理模型、抽樣數量、Few-Shot 與請求間隔等
- 實驗設定與核心程式分離
- 修改實驗條件只需調整 YAML，無須修改程式碼

從資料載入到 Prompt 組裝：確保樣本乾淨、可重現且零洩漏 (src/dataset_loader.py)

題目怎麼抓？Prompt 怎麼拼？

- 資料來源：載入官方標準資料集 cais/mmlu。
- 嚴格切分：
 - 評測題目來自 test 集。
 - 示範範例來自 dev 集。
- 固定抽樣與快取：固定種子 (seed = 42) 確保各模型考題順序完全一致；本地快取避免重複下載。

0-shot 格式

```
The following are multiple choice questions about {subject}.
Question: {question}
A. {choice_A}
B. {choice_B}
C. {choice_C}
D. {choice_D}
Format your output strictly as: 'The correct answer is (X)'
Answer:
```

5-shot 格式

```
The following are multiple choice questions about {subject}.
Question: {dev_q1}
A. ... B. ... C. ... D. ...
Answer: The correct answer is (A)

[... 依序插入 dev 驗證集共 5 題示範 ...]

Question: {test_target_q}
A. {choice_A}
B. {choice_B}
C. {choice_C}
D. {choice_D}
Format your output strictly as: 'The correct answer is (X)'
Answer:
```

AI 審查後的三項關鍵強化

01 提前攔截設定錯誤

啟動時先檢查設定與 Prompt 佔位符

價值 避免輸入模型中途崩潰

02 完整保留分析結構

新增 category，鎖定「領域 → 學科 → 題目」

價值 同一資料支援跨域 / 跨科比較

03 題號、快取與抽樣穩定

{subject}_{index}_{SHA-1} + 固定種子 + 統一快取

價值 不撞名、不重載、跨執行可重現

統一推論介面與多層防護：讓不同模型在同一契約下穩定執行 (src/models/)

多元模型介面與標準格式

多元模型介面 (MODEL PROTOCOL)

- ✓ 本機端 (HuggingFace Pipeline) : huggingface.py
- ✓ 雲端端點 (Gemini API) : gemini_native.py, openai_compatible.py
- ✓ 離線測試樁 (MockModel , 支援固定回覆與隨機種子) :
mock_model.py

超參數設定

Temperature = 0.0

降低生成隨機性，讓相同輸入盡可能產生一致結果。

Max Tokens = 128

限制輸出長度，避免模型產生過長或無關內容。

Top-P = 1.0

不額外截斷候選機率範圍，避免引入額外的採樣限制。

OUTPUT CONTRACT

```
{"question_id", "raw_output", "latency"}
```

AI 審查後的三項關鍵強化

01 Fail-Fast | 啟動前檢查

驗證 max_tokens 、 temperature 、 top_p

→ 避免無效 API 請求與額度浪費

02 Tenacity | 統一重試

429 / 5xx / Timeout → 指數退避，最多 3 次

→ 降低短暫網路異常造成的失敗

03 ERROR | 單題失敗，不讓整份評測中斷

失敗題標記 ERROR，交由 Evaluator 判定無效

→ 單題失敗不影響整體評測

答案抽取與指標計算：杜絕字串誤抓，防止分數虛高 (src/evaluator.py)

三層解析防線與分層指標

TIER 1 嚴格比對 精準匹配 “The correct answer is (A)”



TIER 2 安全回退 優先尋找 answer / choice / option 等提示，仍無法判定
→ 取最後一個獨立 A-D 選項
→ 降低格式差異造成的誤判



TIER 3 安全兜底 衝突、混亂或 ERROR → None；純計算不崩潰

AI 審查後的兩項關鍵強化

01 降低誤判

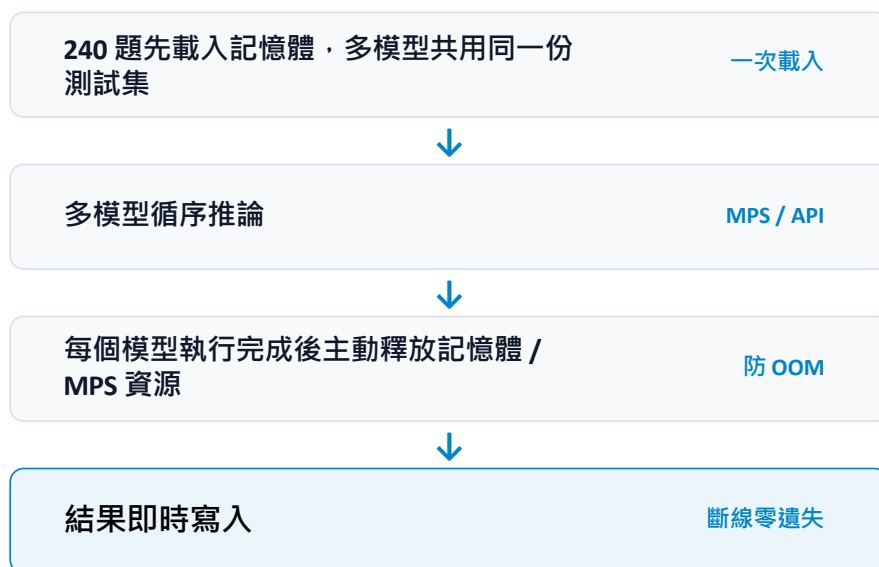
導入單詞邊界，避免把 don't / answer 內字母誤判成 D / A

02 固定分母

Accuracy = Correct / Total；無法判讀視為答錯，杜絕分母縮小

排程管理與系統防護 (src/runner.py + main.py)

單一評估pipeline



main.py | YAML + CLI → 多模型匯總 → 比較報表

AI 審查後的兩項關鍵強化

01 分層防護 | 分清「模型答錯」與「程式寫壞」

網路與模型異常記為無效並繼續跑；資料或程式 Bug 立即中斷報錯

保證 240 題評測跑完，且不把系統缺陷誤算為模型失分

02 專屬對話格式

依模型套用 chat template，重新校對答案擷取起點

降低格式干擾，避免錯判能力

測試流程與品質把關：從單元測試到完整流程，確保結果可靠

兩層測試防線

01 單元測試

293 個案例

4.91 秒 | 全離線 Mock / Stub

02 流程驗收

SMOKE 80 題 | 每科 10 題 | 快速測試

DEMO 240 題 | 每科 30 題 | 正式結果

兩種模式共用同一流程，只調整規模。

品質檢查

工具	檢查項目	結果
ruff check	語法與潛在 Bug	0 違規
ruff format	排版與格式	0 偏差
mypy --strict	型別安全	0 錯誤

```
quote>
• (base) claireweng@wengningxiangdeMacBook-Air mmlu-benchmark % ruff check . && ruff format
--check . && mypy --strict src/

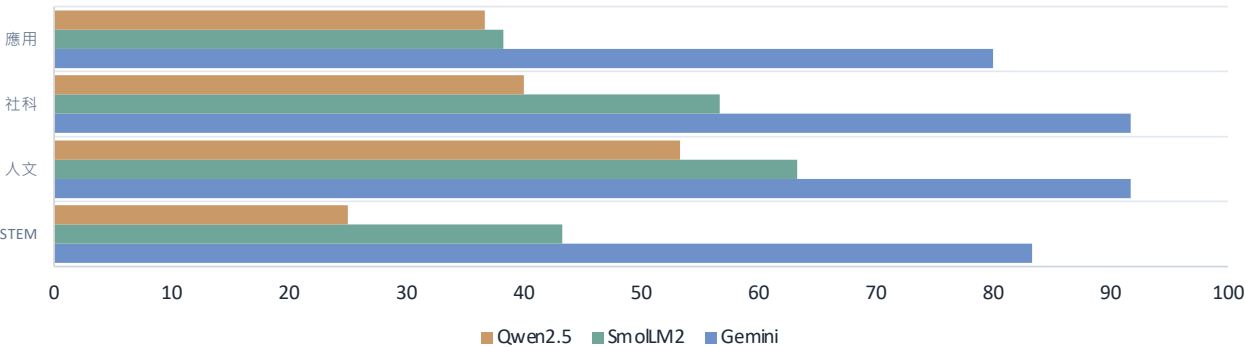
All checks passed!
18 files already formatted
Success: no issues found in 11 source files
(base) claireweng@wengningxiangdeMacBook-Air mmlu-benchmark %
```

評估成果：Gemini 領先；輕量模型在 5-shot 下格式更穩定

跨模型綜合比較

模型	參數量 / 形態	0-shot 正確率	5-shot 正確率	Invalid Rate (0 → 5-shot)	平均延遲 (0 → 5-shot)
Qwen2.5-0.5B	0.5B / 本機 MPS	40.0% (96/240)	38.8% (93/240)	3.8% → 5.4%	0.55s → 0.81s
SmolLM2-1.7B	1.7B / 本機 MPS	49.6% (119/240)	50.4% (121/240)	0.0%	0.92s → 2.23s
Gemini-3.1-Flash-Lite	雲端 API	86.7% (208/240)	86.7% (208/240)	0.8% → 0.0%	7.78s → 2.12s

4 大領域 | 5-shot 正確率 (%)



精確數值

領域	Gemini	SmolLM2	Qwen2.5
STEM	83.3%	43.3%	25.0%
人文 (Humanities)	91.7%	63.3%	53.3%
社科 (Social Sciences)	91.7%	56.7%	40.0%
應用 (Other / Applied Sciences)	80.0%	38.3%	36.7%

AI 工具協同開發歷程：各階段工具配置、流程切換

階段 1 | 文獻查詢與評估流程建置規劃

- 工具：NotebookLM, Gemini, ChatGPT
- 工作內容：整理 MMLU 評估文獻與指標定義，確立資料抽樣、模型、評估指標與全流程設計依據。

階段 2 | 初始架構建立

- 工具：Cursor (Grok 4.6 Medium)
- 工作內容：建立專案目錄結構、eval_config.yaml 參數設定檔雛形，以及初版 dataset_loader.py。

階段 3 | 工具遷移、核心實作與審查

- 工具：VS Code + Cline (Qwen 3.8 27B、Cohere North-mini-code 等)
- 工作內容：
 1. 工具切換：Cursor 免費額度耗盡後，改以 VS Code 搭配 Cline 接續開發。
 2. 模組建置：檢核 dataset_loader.py，並依序完成 runner.py、evaluator.py 與 src/models/ 實作。
 3. 重構與防禦：透過審查提示詞進行多輪審查，完成 8 輪重構並建立 293 筆全離線測試 (4.91 秒完成)。

階段 4 | 技術筆記梳理與報告初稿

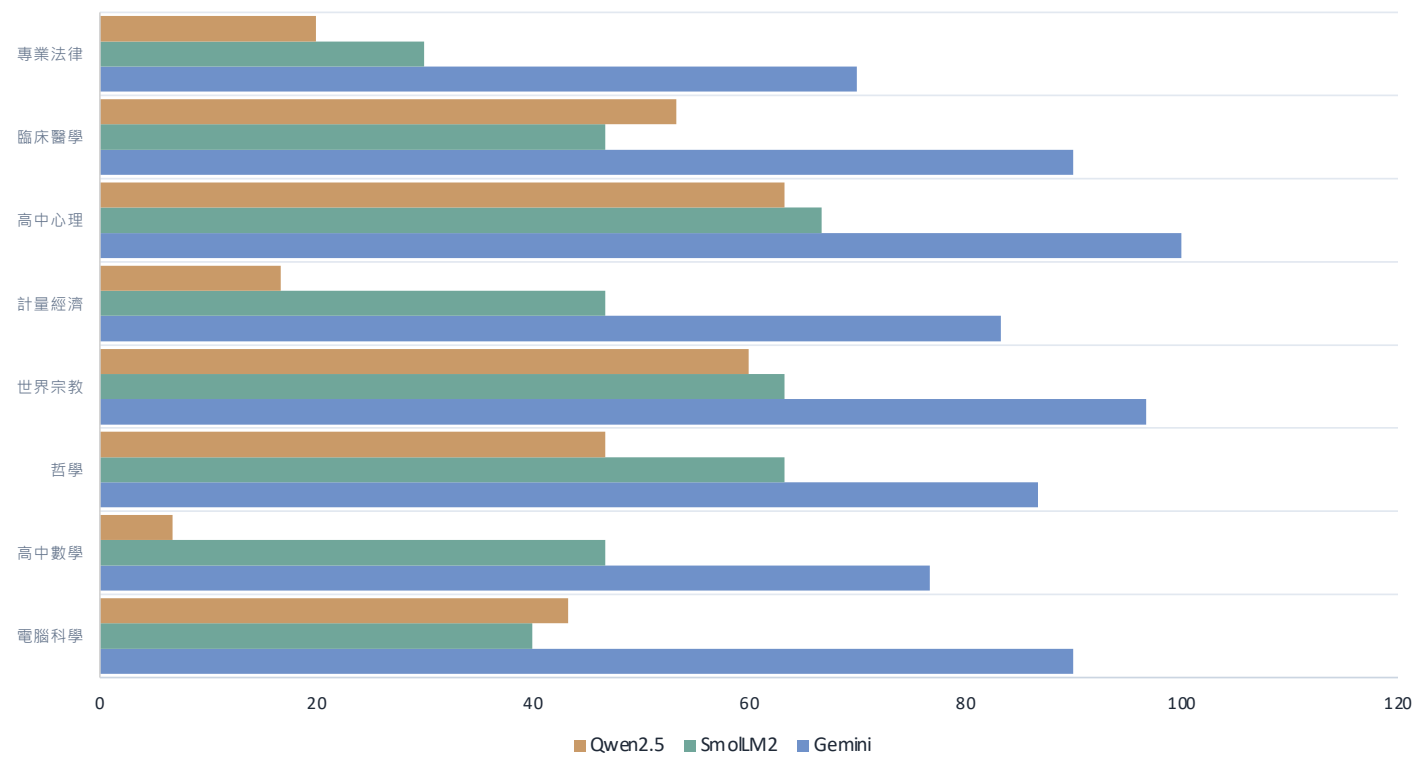
- 工具：Gemini
- 工作內容：將開發歷程記錄由 Gemini 協助歸納為技術報告結構，最終由人工核對技術細節與數據並定稿。

階段 5 | 簡報版面建置

- 工具：Codex
- 工作內容：將專案架構圖、表格與數據洞察轉化為投影片。

八門學科細分：5-shot 顯示模型優勢與弱點集中在不同科目

八門學科正確率 (%)



精確數值

學科	Gem	Smol	Qwen
電腦科學 (college_computer_science)	90.0	40.0	43.3
高中數學 (high_school_mathematics)	76.7	46.7	6.7
哲學 (philosophy)	86.7	63.3	46.7
世界宗教 (world_religions)	96.7	63.3	60.0
計量經濟 (econometrics)	83.3	46.7	16.7
高中心理 (high_school_psychology)	100.0	66.7	63.3
臨床醫學 (clinical_knowledge)	90.0	46.7	53.3
專業法律 (professional_law)	70.0	30.0	20.0

單位：% | 5-shot | 每科 30 題